Rapport för inlämningsuppgift till Kurs 05:
Algoritmer, Datastrukturer och Design Patterns

github.com/maxrangner/IoT-25/tree/main/05-algoritmer-datastrukturer-och-design-patterns/inlamning

ADTer / Datastrukturer:

- struct Event - Struct som håller data från sensorerna. Den håller sensorId för att hålla koll på alla sensorer, sensorType för att hantera värdet från sensorn på rätt sätt, tidsstämpel som sätts vid skapande av Event, och slutligen ett värde som avlästs. Events kopieras mellan producer, kö och logg för att tydligt separera ägandeskap och undvika beroenden till temporära variabler.

- EventQueue / Ringbuffer - Denna cirkulära kö är definierad som en array på 20 platser av typen Event. Den skapas i main() och läggs på stacken eftersom dess storlek är fast och livslängden är hela programmets körning. Detta förenklar minneshantering och undviker onödig heap-allokering. Minnet för den släpps automatiskt vid programslut, vilket sker när quit() avslutar while-loopen i main() och programmet terminerar.

- EventLog / Länkad lista - Loggen, där datan kommer leva längst, implementeras som en länkad lista där varje ny nod (Event) allokeras minne genom malloc() och lever därefter på heapen. Vid quit() kallas logDestroy som traverserar hela listan och frigör minnet genom free().

- TriggeredAlarms / Mängd - Håller koll på vilka sensorId som vid senaste läsning triggade sina respektive satta larm genom att vara över ett tröskelvärde.

Funktioner:

- Main() - Skapar kön, loggen och alarm-mängden. Skapar även en struct typedefad Context, som håller pekare till samtliga datastrukturer som används av programmet. Efter det går main() in i en while-loop som körs tills funktionen quit() sätter en bool isRunning till false, vilket kör relevanta minneshanteringsfunktioner och sedan bryter loopen och avslutar programmet.

Design Patterns:

- Strategy - Används vid val av sorteringsalgoritmer. En funktionspekare sparas som sortFunction. Funktionen selectSort tar in argument av önskad sortering och returnerar rätt algoritm. Vi sparar sedan ner vår logg till en tillfällig array som sorteras och printas ut till gränsnittet.

- Dispatch - Meny som tar in användarkommandon, parsar och returnerar korrekt funktion.

- Observer - Efter att ett Event processas av consumer kallas notify() som vidarebefordrar Event till två observers (logger och alarm) via funktionspekare.

Tidskomplexitet:

- Bubblesort - Bästa utfall O(n) och värst O(n^2) om vi tvingas bubbla igenom hela listan till slut.

- Selection Sort - Alltid O(n^2).

- Linjär sökning - I detta program söks Events efter sensorId, vilket kräver att hela loggen traverseras och därför alltid ger O(n). Om sökningen istället gällt ett unikt värde hade bästa fall varit O(1) och värsta O(n).

Flow:

- tick() - Kör producer() <n> antal gånger. Efter det kör den consumer() tills EventQueue är tom.
- producer() - Genererar vid varje kall ett Event per unik sensor (tre stycken) och placerar i eventQueue() via enqueue().
- consumer() - Plockar ut första elementet i kön med dequeue() och skickar sedan Event till notify().
- notify() - Kallar på sina två registrerade funktionspekare (logger och alarm). Logger använder logAppend för att slutligen lägga Event i loggen.